DEV WEEKENDS · TECH TALK

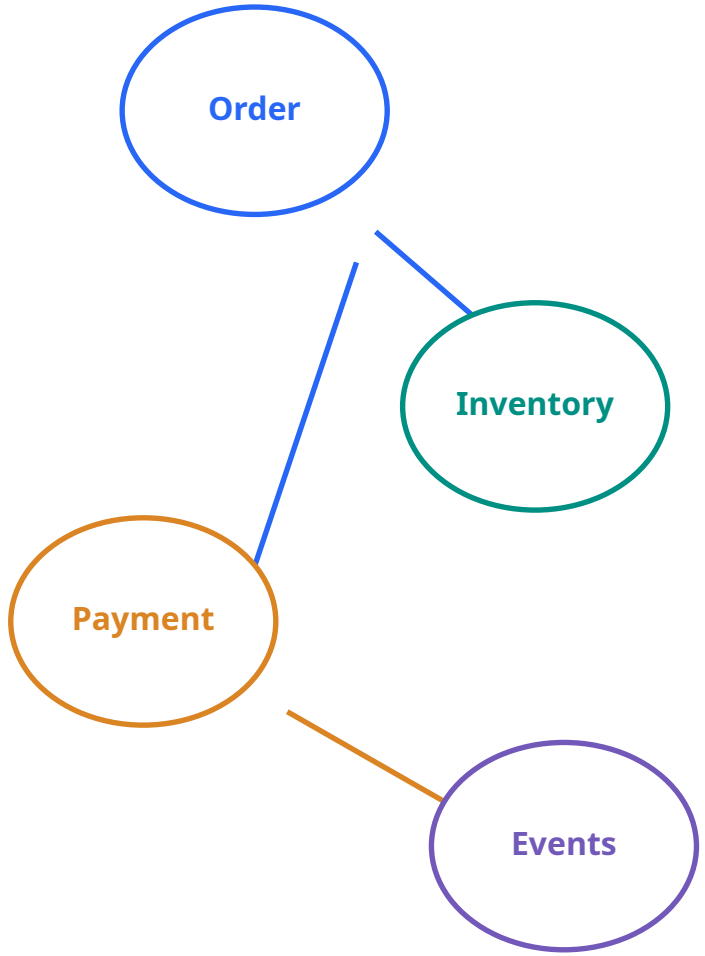
Beyond Microservices

Real Engineering Challenges

Real distributed-system failures, their causes, and engineering solutions

From lost payment replies to retry storms, stale locks, safer releases, and security boundaries.

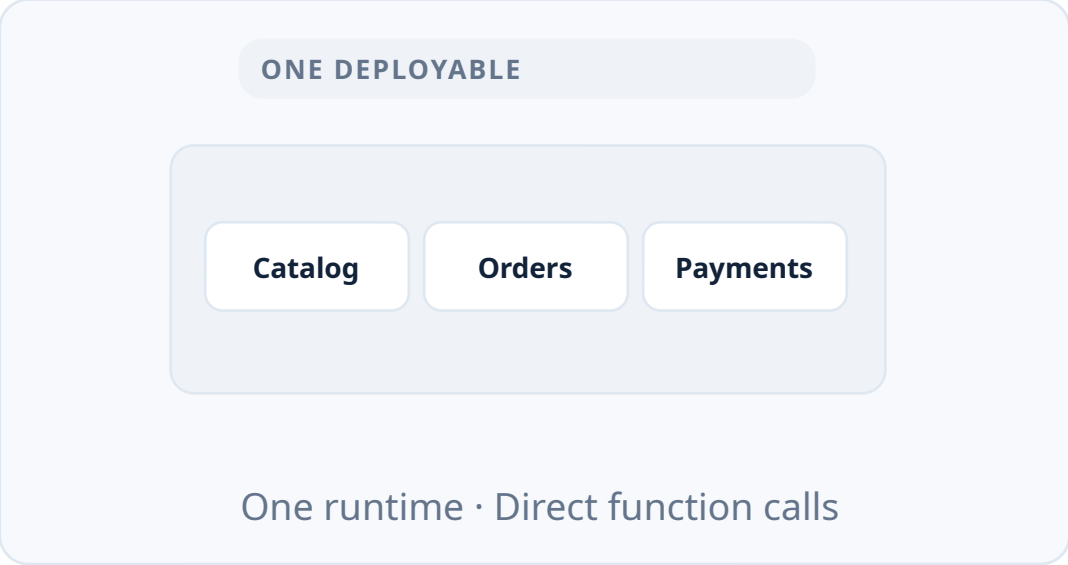
80-minute engineering talk + 10-minute Q&A

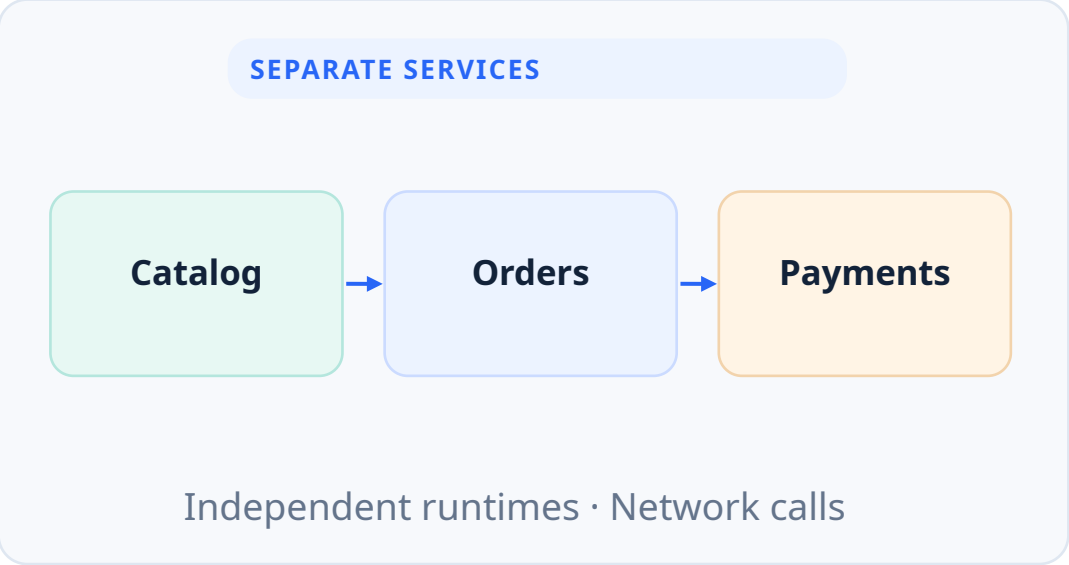


ALI SHAIR

One application becomes many independent services

The architecture changes when local calls become network requests.





Monolith

A modular monolith can still be well-structured and scalable.

Microservices

Services are independently runnable and may deploy at different times.

Freedom to deploy also creates failure boundaries

Distribution is a trade-off, not a free performance or reliability upgrade.

WHAT WE GAIN

Independent releases and scaling. Teams can evolve services separately when boundaries are well designed.

WHAT WE ADD

Network latency, partial failure, uncertain remote outcomes, harder data coordination and operating complexity.

GUIDING QUESTION

What breaks when a function call becomes a network request?

Good default: start modular; distribute only where the benefits justify the cost.

Two ways services collaborate

Synchronous calls need a result; asynchronous messages let other work happen later.

FOUNDATIONS

SYNCHRONOUS

Order

Caller

→

Inventory

Responds

Caller waits for the response before that operation continues.

ASYNCHRONOUS

Order

Event

→

Broker

Queue / log

→

Worker

Later

Publishing is separate from the consumer finishing its work.

Important distinction

JavaScript async/await can still be a synchronous business dependency.

Commands vs events

ReduceStock requests an action; OrderCreated reports a fact.

REST and gRPC serve different API needs

Both cross the network, and both can fail or time out.

REST — HTTP RESOURCE APIs

Routes such as GET /products and POST /orders. Friendly to browsers, tools, external integrations and gateways.

gRPC — REMOTE METHODS

A service exposes defined operations such as ReduceStock(productId, quantity). Useful for controlled internal APIs.

Choose by contract

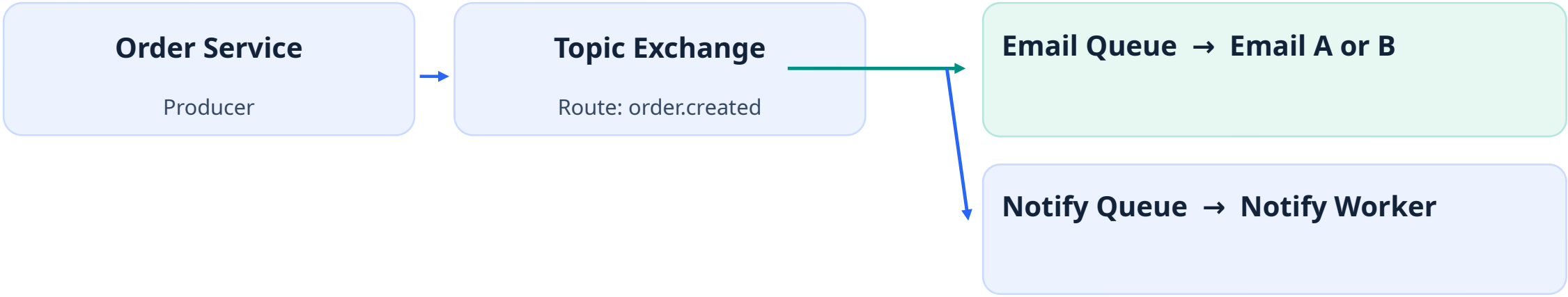
Client requirements and API contracts matter more than hype.

Still a network call

A gRPC method may resemble a local function, but it can be slow or uncertain.

RabbitMQ routes work through exchanges and queues

One order event can trigger independent jobs, while workers share a queue.



Exchange + routing key

The exchange directs published messages into bound queues.

Competing consumers

Email A and B share one email queue; only one handles each delivery.

Acknowledgment

Consumer acks can be lost; redelivery can repeat a business effect.

Kafka stores events in ordered partitions

Partitions distribute records; offsets mark positions within each partition.



Ordering

Order is guaranteed within a partition, not across the whole topic.

Record key

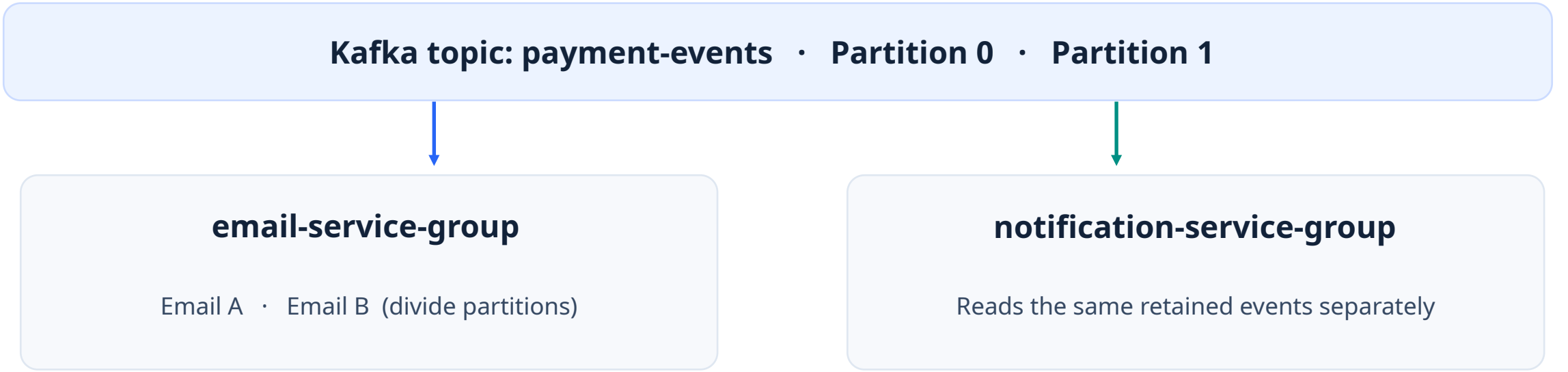
A stable orderId key can help related records use the same partition.

Retention

Reading does not delete the record; it remains until retention removes it.

Kafka consumer groups read events independently

Members share partitions within one group; different groups track separate progress.



Committed offset

Records each group’s progress within a partition.

Replay

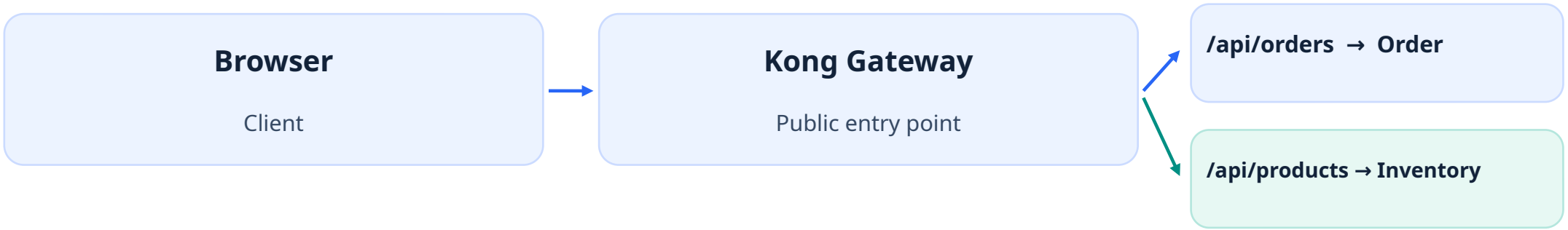
A group can reread retained events from earlier positions.

Consumer lag

Growing lag means processing falls behind produced records.

An API Gateway keeps public routing in one place

Kong routes incoming requests; internal services remain separate.



Routing

The client uses a stable public API rather than service-specific internal ports.

Load balancing

An upstream can distribute HTTP requests across available instances.

Rate limiting

Configured policies can reject excess requests (often HTTP 429).

Gateway routing and Kafka/RabbitMQ messaging solve different communication problems.

The Challenges Begin

Real failures. Clear causes. Practical engineering solutions.



Follow each letter: Challenge A → Solution A. When several approaches help, see A1, A2, etc.

Changing inventory data breaks the order service

Two services use the same database field. Inventory renames it, and Orders starts failing.

WHAT HAPPENS

1 · BEFORE

Orders reads

products.stock

→

2 · CHANGE

Inventory renames

stock → available_qty

→

3 · FAILURE ✕

Orders still reads stock

That field no longer exists

CAUSE

Order Service depends directly on Inventory’s private database structure.

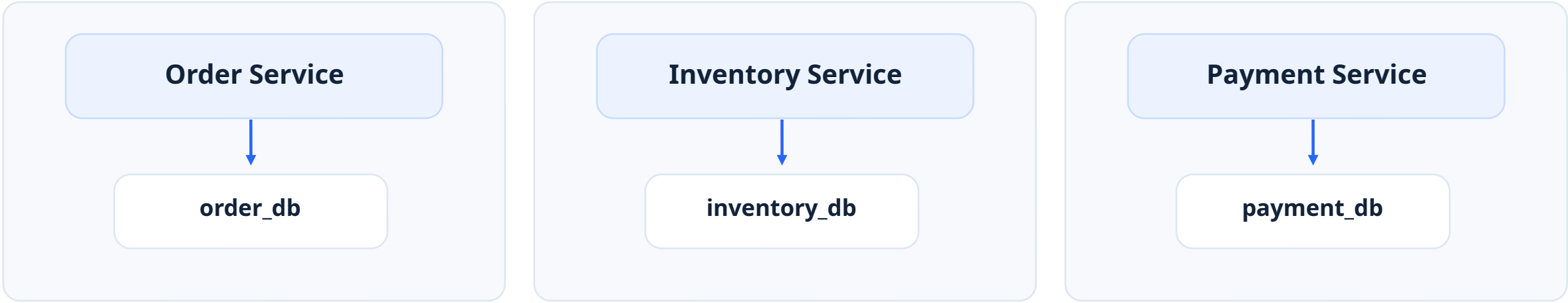
IMPACT

A normal Inventory update can break checkout without changing Order Service.

Database per service protects data ownership

Each service controls its own schema and exposes operations through an API.

SOLUTION A



Logical ownership

Separate database names can share one physical MongoDB server in the demo.

Boundary rule

Other services use the owner’s API instead of directly writing its tables.

Trade-off

Cross-service queries and multi-service updates become more complex.

The order is saved, but the payment fails

Checkout spans three databases. Two steps finish, and the last step fails.

WHAT HAPPENS

1 · SUCCESS

Order database

Order saved

→

2 · SUCCESS

Inventory database

Stock reduced

→

3 · FAILURE

Payment service

Payment cannot complete

CAUSE

Each service commits its own database changes independently.

IMPACT

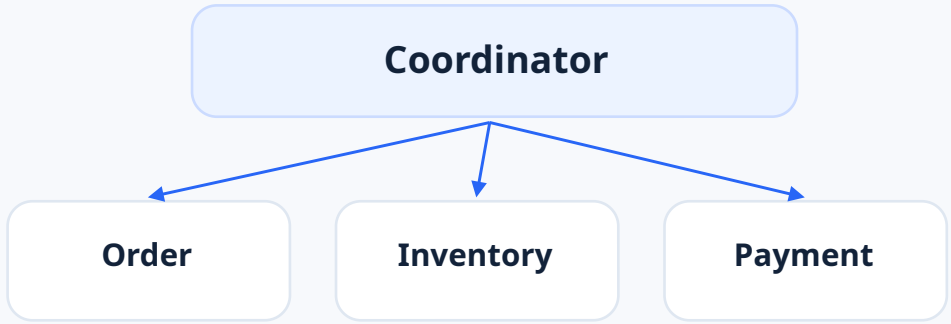
One local rollback cannot reverse commits that another service already finished.

Saga coordinates the steps of a distributed workflow

A business transaction becomes a sequence of local transactions.

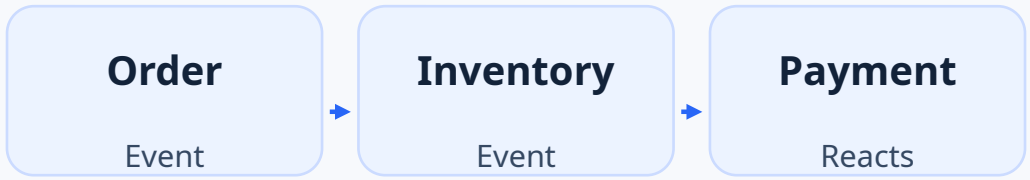
SOLUTION B

ORCHESTRATION



Central component directs the sequence

CHOREOGRAPHY



Services react to each other’s events

Local transactions

Each participant commits changes in the database it owns.

Design trade-off

Orchestration centralizes flow; choreography spreads coordination logic.

Failure paths

A Saga needs explicit recovery decisions when a local step fails.

The order is created, but the dashboard still says pending

The order database has new data. A separate dashboard has not received the update yet.

WHAT HAPPENS

1 · NEW DATA

Order database

Status: Created

→

2 · DELAY

Update event

Still being delivered

→

3 · OLD VIEW ×

Dashboard

Status: Pending

CAUSE

The dashboard uses a separate view that is updated after the order database.

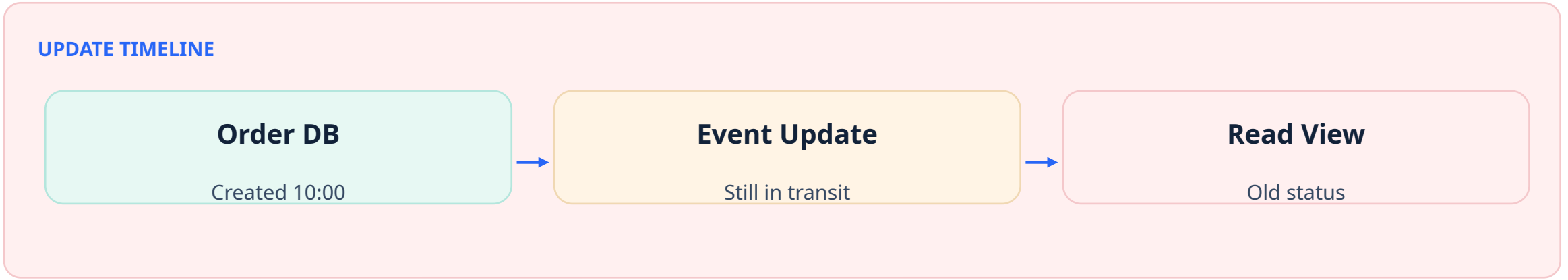
IMPACT

A user can briefly see older information even though the order was created.

Eventual consistency: handle a delayed read view

Read views may briefly lag; display honest pending states while updates arrive.

SOLUTION C



Eventual consistency

Views can briefly differ and converge once updates are successfully processed.

Design for lag

Show pending/processing states; avoid claiming all work is complete.

Use appropriate consistency

A dashboard can tolerate lag; critical financial invariants may not.

The payment succeeds, but the response disappears

The customer was charged. Order Service sees a timeout and may try the same payment again.

WHAT HAPPENS

1 · REQUEST

Order Service

Charge Rs. 2,000

→

2 · SUCCESS

Payment Service

Customer charged

→

3 · LOST REPLY

Network connection

Order never gets result

CAUSE

A timeout tells the caller it stopped waiting, not whether the charge succeeded.

IMPACT

Retrying the same purchase can charge the customer twice.

Idempotency protects a repeated business action

The same logical payment must not create duplicate charges.

SOLUTION D

First request

Key: pay-101

→

Payment Service

Charge once

→

Retry with key

Return stored result

Same idempotency key on every retry of ONE logical operation

Persist the result

Keep a durable record that a logical request was already processed.

Atomic duplicate protection

Concurrent retries must not both pass a check-then-charge race.

Check request details

Reuse a key only for the same action and parameters.

An email is sent, then the same job arrives again

The worker finishes sending the email but crashes before confirming the message.

WHAT HAPPENS

1 · RECEIVE

Email worker

Gets OrderCreated

→

2 · SUCCESS

Email worker

Sends confirmation

→

3 · REDELIVERY ✕

Broker

No ACK was received

CAUSE

The broker cannot tell that processing finished if the worker never acknowledged.

IMPACT

Another worker may send the same email a second time.

Delivery semantics: protect against duplicate effects

Message brokers may redeliver work; the application must handle repeats safely.

SOLUTION E

Event processed → Worker crashes before ACK → Broker redelivers

AT-MOST-ONCE

May lose work; a message is not redelivered.

AT-LEAST-ONCE

Redelivery reduces loss risk but can repeat a side effect.

EXACTLY-ONCE EFFECT

Duplicate delivery must not repeat the protected business effect.

RabbitMQ uses acknowledgments; Kafka uses committed offsets. Neither alone prevents every duplicate business action.

Two database replicas lose their connection

Both database copies are running, but they cannot exchange new updates.

WHAT HAPPENS

1 · NORMAL

Replica A & B

Both can communicate

→

2 · BROKEN LINK

Network partition

A cannot reach B

→

3 · REQUESTS

Clients

Ask both sides for data

CAUSE

The replicas cannot agree on new writes while communication is broken.

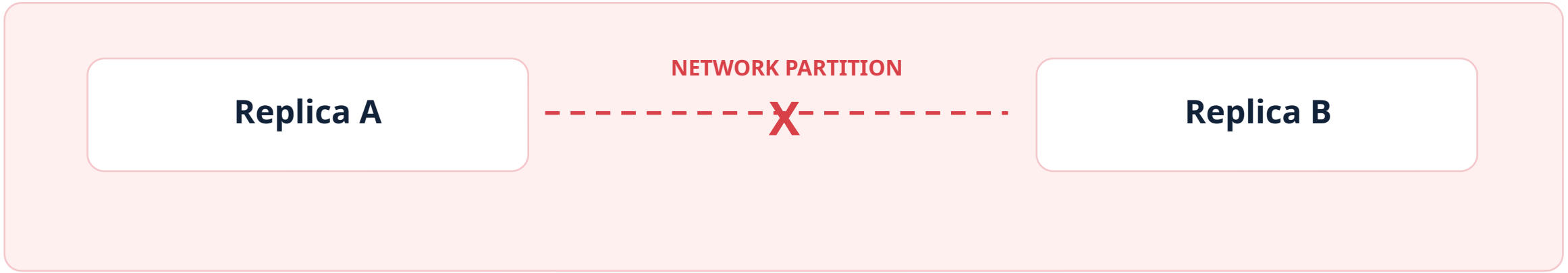
IMPACT

A side may refuse some requests to stay consistent, or sides may temporarily disagree.

CAP: consistency and availability during a partition

Separated replicas must make a trade-off when they cannot coordinate.

SOLUTION F



CONSISTENCY-FIRST

Some operations wait or fail rather than return conflicting results.

AVAILABILITY-FIRST

Separated sides may continue responding but disagree temporarily.

THE CAP INSIGHT

The forced trade-off appears during a partition, not all the time.

Three workers run the same billing job

All three instances start the monthly billing task at the same time.

WHAT HAPPENS

1 · SCHEDULE

Billing A

Starts billing

→

2 · SAME TIME

Billing B

Also starts billing

→

3 · DUPLICATE ×

Billing C

Also starts billing

CAUSE

A mutex inside one instance is invisible to instances on other machines.

IMPACT

The same customer may receive multiple charges for one billing cycle.

A shared lock gives one worker a temporary lease

Workers compete for ownership through shared coordination infrastructure.

SOLUTION G



Shared authority

Workers consult one coordinator instead of independent local mutexes.

Lease expiry

If a worker dies, ownership can eventually be granted again.

Not a complete guarantee

A paused worker may resume after its lease has expired.

The next challenge appears when an old owner wakes up after the lease expires.

The old worker continues after its lock expires

Worker A pauses. Its lease expires, Worker B takes over, and A suddenly resumes.

WHAT HAPPENS

1 · PAUSE

Worker A

Stops responding

→

2 · TAKEOVER

Worker B

Gets the new lease

→

3 · OVERLAP

Worker A wakes up

Both may try to write

CAUSE

A lease can expire without physically stopping the previous worker.

IMPACT

An old worker may overwrite newer work even though it no longer owns the lock.

Fencing tokens reject stale workers

Each new lock owner receives a higher number that the destination validates.

SOLUTION H

OLD OWNER

Worker A

Fencing token 41

CURRENT OWNER

Worker B

Fencing token 42

Increasing tokens

New leases receive monotonically increasing ownership generations.

Destination check

The protected storage system rejects old tokens after accepting newer ones.

Keep idempotency

Fencing helps with stale writes; it does not replace payment-level duplicate protection.

The payment happened later, but its timestamp is earlier

Different computers disagree on time, so sorting timestamps shows the wrong order.

WHAT HAPPENS

1 · FIRST

OrderCreated

Clock A: 10:00:00.100

→

2 · LATER

PaymentProcessed

Clock B: 10:00:00.080

→

3 · WRONG ORDER ✕

Logs sorted by time

Payment appears first

CAUSE

Machine B’s clock is behind Machine A’s clock.

IMPACT

Wall-clock timestamps alone cannot reliably show which event caused another.

Lamport clocks help order related events

Logical counters track happens-before relationships despite clock differences.

SOLUTION I

OrderCreated

Machine A · 10:00:00.100

PaymentProcessed

Machine B · 10:00:00.080

LOGICAL CLOCK SOLUTION

Send a logical counter with the request; receiver uses max(local, received) + 1.

If A caused B, then L(A) < L(B). The reverse alone does not prove causation.

A slow payment service makes checkout requests pile up

Order Service keeps waiting for Payment Service while more customers place orders.

WHAT HAPPENS

1 · CUSTOMER

Checkout requests

Keep arriving

→

2 · WAITING

Order Service

Pending calls increase

→

3 · BOTTLENECK

Payment Service

Responds very slowly

CAUSE

Calls remain in progress while the dependency is delayed.

IMPACT

Memory, connections or concurrency limits can fill and affect other requests.

Timeouts put an upper bound on waiting

The caller stops waiting after its deadline; remote outcome may still be unknown.

SOLUTION J1



Set a deadline

Choose a budget from observed latency and caller requirements.

Cancellation is limited

Aborting a client call cannot undo a remote commit.

Timeout ≠ failure proof

The payment may have succeeded before the reply was lost.

Temporary failures deserve careful retries

Backoff gives the dependency time to recover; jitter prevents synchronized bursts.

SOLUTION J2



Bounded retries

Limit total attempts and honor an overall deadline.

Exponential backoff

Each retry waits longer instead of immediately repeating.

Jitter

Add randomness to prevent many clients retrying together.

Retry only transient failures, and repeat side effects only when they are safe.

Retries make an overloaded service even busier

Two hundred requests each retry twice. The failing dependency may receive 600 calls.

WHAT HAPPENS

1 · LOAD

200 customer requests

Hit a struggling service

→

2 · RETRIES

Three total attempts

For each request

→

3 · MORE LOAD

Up to 600 calls

Pressure triples

CAUSE

Failing requests create more downstream attempts while the dependency is struggling.

IMPACT

A temporary slowdown can turn into a longer outage or retry storm.

Retry budgets control the extra load

Each recovery attempt must justify the pressure it places on a dependency.

SOLUTION K1

Retry only select errors

Transient 503/network faults may recover; permanent bad input will not.

Backoff + jitter

Spread attempts instead of creating synchronized spikes.

Overall deadline

Stop retries when the user request has no useful remaining budget.

Retry budget

Cap retries relative to normal traffic or allowed capacity.

Idempotent effects

Make repeated payment/stock operations safe at the receiver.

Circuit breaker

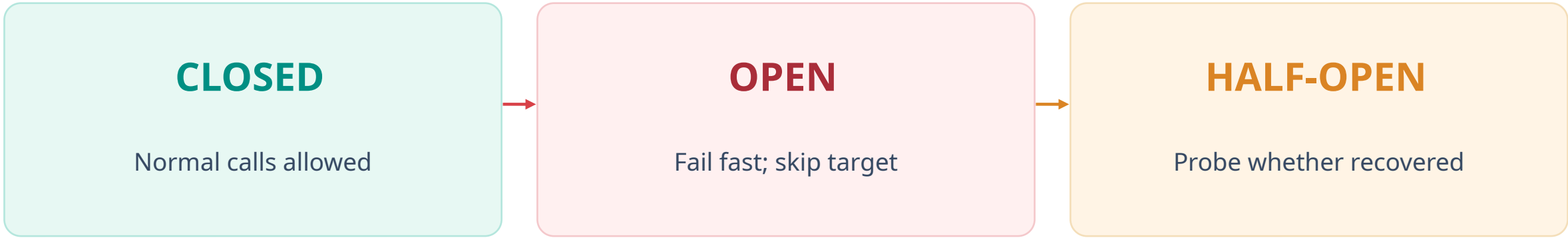
Stop normal calls when repeated failures show a dependency is unhealthy.

Retries help transient failure. Persistent failure needs a different response.

A circuit breaker stops calling a failing dependency

It remembers recent results and switches between three operational states.

SOLUTION K2



Failure policy

Rolling failure rate and minimum request count determine opening.

Cooldown

An open breaker waits before allowing a recovery probe.

Recovery

Successful probe closes; failed probe reopens the circuit.

Watch the breaker fail fast, then recover

Resilience Lab · Opossum · Target Service

SOLUTION K2

01 **HEALTHY**

Target responds normally

Closed

02 **INJECT FAILURE**

Stop target container

Failures observed

03 **OPEN**

Protected calls fail fast

No normal downstream call

04 **RECOVER**

Start target; allow probe

Half-open → Closed

docker compose stop resilience-target-service | docker compose start resilience-target-service

A slow recommendation service blocks payment traffic

Recommendation calls consume the shared capacity that healthy payment calls need.

WHAT HAPPENS

1 · SLOW

Recommendations

Requests remain open

→

2 · FULL

Shared request slots

No free capacity

→

3 · BLOCKED

Payment calls

Cannot get a slot

CAUSE

Unrelated dependencies compete for the same limited connections or concurrency.

IMPACT

One optional slow feature can interfere with critical payment operations.

Bulkheads isolate concurrency so failures stay contained

Cap concurrent calls to one dependency and reject excess work quickly.

SOLUTION L

5 simultaneous requests · Limit = 2

Request 1	Request 2	Request 3	Request 4	Request 5
ADMITTED	ADMITTED	REJECTED	REJECTED	REJECTED

Bounded capacity

Two calls enter; three receive fast rejection instead of piling up.

Isolation matters

Separate allocations prevent one dependency consuming every slot.

Demo limitation

The in-memory counter limits one Lab instance, not an entire cluster.

Recommendations are down, so the whole product page fails

The core product details work, but an optional recommendation call is unavailable.

WHAT HAPPENS

1 · PAGE

Customer opens product

Product data is ready

→

2 · FAILURE

Recommendation service

Does not respond

→

3 · TOO MUCH

Entire page errors

Customer cannot shop

CAUSE

The page treats an optional feature as required for the whole response.

IMPACT

Customers lose a useful core feature because a nonessential service failed.

Graceful degradation keeps core features working

Optional failures should not prevent essential and correct user actions.

SOLUTION M

OPTIONAL FEATURE FAILS

Recommendations are unavailable. The product page still displays price, images and availability.

ESSENTIAL OPERATION FAILS

Payment is unavailable. Checkout must not claim the customer paid successfully.

Safe fallback

Hide or replace optional content with a known-good alternative.

Preserve correctness

A fallback must never fake a successful financial operation.

Traffic spikes beyond the service’s capacity

Even well-behaved clients can create more total work than a service can handle.

WHAT HAPPENS

1 · NORMAL

Service capacity

Handles ordinary traffic

→

2 · SURGE

Many clients

Arrive at once

→

3 · OVERLOAD

Work queue grows

Latency and failures rise

CAUSE

The sum of incoming demand exceeds available processing resources.

IMPACT

Accepting every request can slow or crash the service for everyone.

Rate limiting and load shedding protect capacity

Usage policies and overload controls solve different traffic problems.

SOLUTION N

RATE LIMITING

A policy controls request volume for a client, endpoint, or tenant. Excess requests often receive HTTP 429.

LOAD SHEDDING

When total capacity is exhausted, reject or drop selected work to protect the service and its core traffic.

Not the same trigger

All clients can obey their limits while aggregate demand still overwhelms the service.

Fail predictably

Controlled rejection is safer than unbounded queues and complete collapse.

Our resilience code looks correct—until a real outage

Timeouts and breakers are configured, but nobody has tested what happens when the dependency disappears.

WHAT HAPPENS

1 · ASSUMPTION

Developers

Expect graceful failure

→

2 · OUTAGE

Dependency stops

Calls start failing

→

3 · SURPRISE

System behavior

Recovery is untested

CAUSE

Design diagrams and successful normal requests do not prove failure behavior.

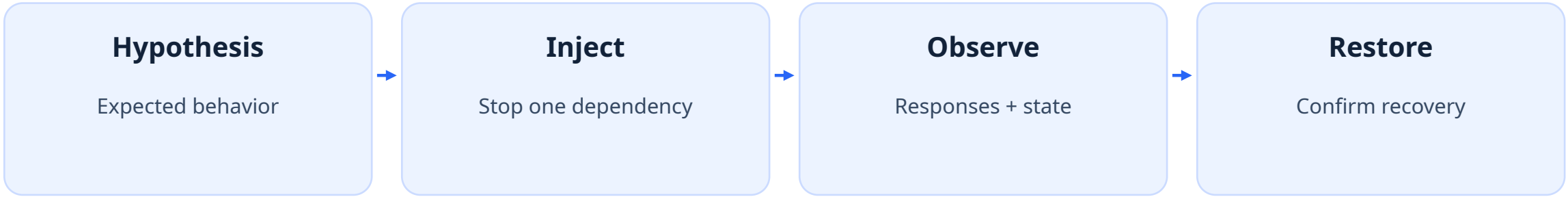
IMPACT

An outage may reveal that timeouts, circuit breakers or recovery logic do not work as expected.

Chaos engineering tests failure and recovery

Controlled experiments reveal whether resilience patterns work as expected.

SOLUTION O



Start with small blast radius

Restrict which service, users and environment can be affected.

Define success first

Know what healthy degradation and recovery should look like.

A demo is evidence, not proof

One successful experiment does not guarantee all failures are safe.

An Order Pod crashes and leaves less capacity

The application needs three instances, but only two are running after one fails.

WHAT HAPPENS

1 · HEALTHY

Order Service

Three Pods running

→

2 · CRASH

One Order Pod

Stops unexpectedly

→

3 · GAP

Remaining Pods

Only two are available

CAUSE

Application processes and containers can exit unexpectedly.

IMPACT

Without automated recovery, the service runs below its intended capacity.

Kubernetes reconciles desired and actual replicas

Deployments maintain Pods across failures and rolling updates.

SOLUTION P

DESIRED STATE: 3 healthy Order Service Pods

Order Pod 1

Order Pod 2

Order Pod 3

Pod

Smallest deployable unit, typically with one application container.

Deployment

Controller declares replica count and manages replacement/update through ReplicaSets.

Reconciliation

Kubernetes continuously works to close the gap between actual and desired state.

A replacement Pod has a new IP address

Inventory recovers in a new Pod, but Order Service still calls its old address.

WHAT HAPPENS

1 · BEFORE

Inventory Pod

IP: 10.2.1.10

→

2 · REPLACED

New Inventory Pod

IP: 10.2.1.25

→

3 · WRONG TARGET ✕

Order Service

Calls 10.2.1.10

CAUSE

Individual Pod addresses can change when instances are replaced.

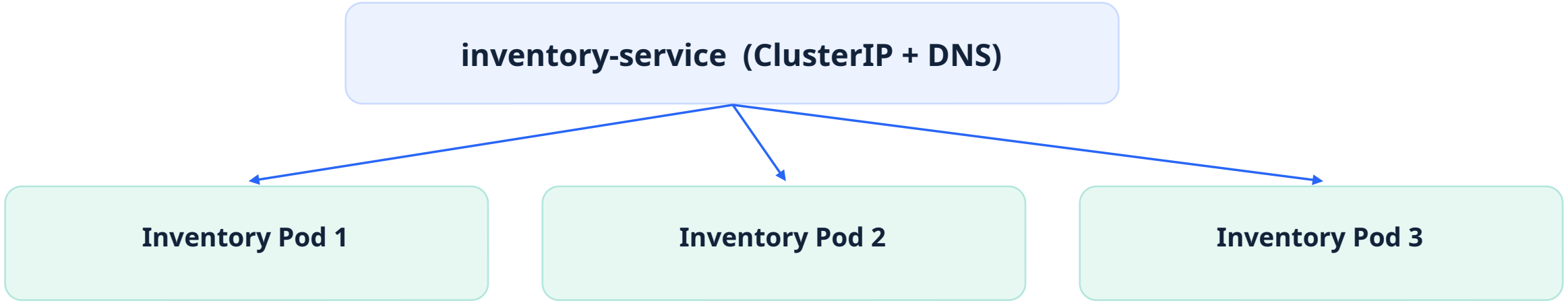
IMPACT

Hardcoded Pod addresses cause broken service-to-service calls.

Kubernetes Services provide stable access to Pods

Call a Service name instead of relying on an individual Pod IP.

SOLUTION Q



Stable discovery

Call the Service name rather than a changing individual Pod IP.

Readiness probe

An unready Pod should not receive normal Service traffic.

Liveness probe

Detect an unrecoverably stuck container that should be restarted.

A compromised internal service calls the payment API

An attacker controls one worker inside the cluster and attempts a sensitive payment operation.

WHAT HAPPENS

1 · COMPROMISED

Worker container

Attacker gains control

→

2 · REACHABLE

Payment endpoint

Inside same network

→

3 · ATTEMPT ✕

Sensitive API

Unauthorized request

CAUSE

Being on a private network does not prove workload identity or permissions.

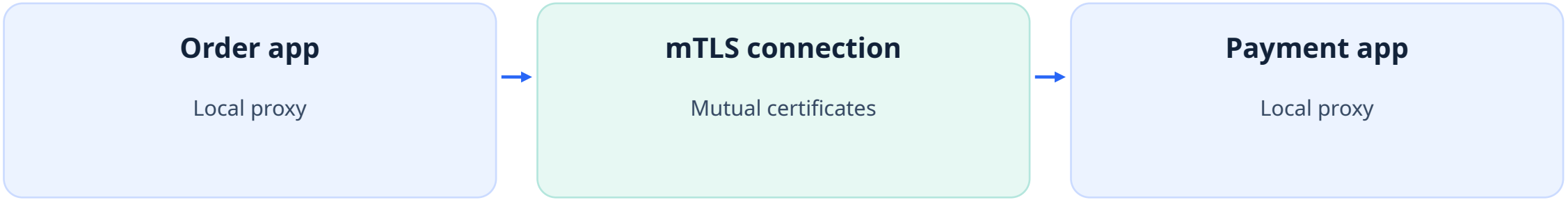
IMPACT

Payment Service needs to identify its caller and check whether the action is allowed.

mTLS verifies both sides of a service connection

A service mesh can apply identity and traffic policies consistently.

SOLUTION R



TLS vs mTLS

TLS normally authenticates the server; mTLS also authenticates the client.

Authentication vs authorization

Knowing the caller identity is separate from allowing a payment operation.

Service mesh

Istio/Linkerd can manage identities and traffic via proxies or other data planes.

The new release passes health checks but breaks checkout

Order Service v2 starts normally, but calculates customer discounts incorrectly.

WHAT HAPPENS

1 · DEPLOY

Order v2

Returns HTTP 200

→

2 · HIDDEN BUG

Discount logic

Calculates wrong total

→

3 · IMPACT

Customers

Receive wrong prices

CAUSE

Process health checks cannot validate every business rule.

IMPACT

Sending all customers to an unproven release exposes everyone to the bug.

Rolling, blue-green and canary reduce rollout risk

Each controls who receives a new version and how quickly exposure grows.

SOLUTION S

ROLLING

Gradually replace old Pods with new Pods; versions can coexist.

BLUE-GREEN

Prepare two environments and switch traffic after validation.

CANARY

Start with a small traffic slice (e.g. 5%), then expand if healthy.

A fast traffic rollback is not a database rollback.

Old and new API/database versions must remain compatible while traffic overlaps.

Each service passes tests, but checkout still fails

Order, Inventory and Payment work alone. The complete request flow breaks when they run together.

WHAT HAPPENS

1 · PASS

Order tests

Business logic works

→

2 · PASS

Payment tests

Business logic works

→

3 · FAIL

Complete checkout

Services disagree

CAUSE

Tests of isolated code may miss mistakes in APIs, databases and cross-service behavior.

IMPACT

The user journey fails even when individual service tests are green.

Unit, integration and end-to-end testing

Each test level checks a different part of a distributed application.

SOLUTION T

UNIT TEST

Validate calculations and business logic in isolation.

INTEGRATION TEST

Exercise real database, messaging or service integration.

END-TO-END TEST

Verify critical user workflows across services.

No single test type proves the whole distributed system is safe.

Inventory renames a field and breaks Order Service

Inventory changes an API response, but Order Service still expects the old field name.

WHAT HAPPENS

1 · EXPECTED

Order Service

Reads {stock: 20}

→

2 · UPDATED

Inventory API

Sends {quantityAvailable: 20}

→

3 · FAILURE

Order Service

Cannot read stock

CAUSE

One service changes an interface that another service depends on.

IMPACT

A successful provider deployment can break consumers without changing their code.

Consumer-driven contract tests catch API breaks

Verify that a provider’s changes preserve what its consumers expect.

SOLUTION U

Order expects

{ stock: 20 }

Inventory now returns

{ quantityAvailable: 20 }

Consumer expectation

Order specifies the response structure it depends on.

Provider verification

Inventory verifies that its current API satisfies the contract.

Safer releases

Detect incompatible changes before affected services deploy.

The happy-path tests pass, but recovery is broken

Checkout succeeds when every service works. Nobody tested a timeout or dependency outage.

WHAT HAPPENS

1 · HAPPY PATH

All services

Checkout succeeds

→

2 · FAILURE

Payment unavailable

Request times out

→

3 · UNTESTED

Recovery behavior

Bad retry or error state

CAUSE

Normal tests never exercise dependency failure and recovery conditions.

IMPACT

A release can pass basic checks yet fail badly when a service goes down.

Failure tests and controlled release validation

Test outages and retries; introduce changes to a limited audience first.

SOLUTION V

FAILURE TESTS

Stop a dependency; verify timeouts, safe retries, error states and recovery.

FEATURE FLAGS

Enable a new feature for selected users without a full redeploy.

CANARY VALIDATION

Check real error and business outcomes before expanding exposure.

Test the failure mode you designed for, not only the success path.

Distributed systems are engineering trade-offs

Each solution controls a failure mode and introduces costs of its own.

01	**COMMUNICATION**	Every network call introduces uncertainty.
02	**DATA**	Local success does not imply global correctness.
03	**RESILIENCE**	Contain failure instead of assuming it will not happen.
04	**DEPLOYMENT & SECURITY**	Instances and versions change; identity and compatibility matter.
05	**TESTING**	Prove the response to real failure conditions.

Understand the failure first. Then choose the mechanism that fits.

DEV WEEKENDS · THANK YOU

Questions?

Which distributed failure would you design for first?

ALI SHAIR

Find the project, or connect after the talk.

PORTFOLIO	ali-shair-portfolio.netlify.app
GITHUB	github.com/alishair7071
LINKEDIN	linkedin.com/in/swe-ali-shair
EMAIL	alishair7071@gmail.com

github.com/alishair7071/microservices-all-in-one